IEEE-STYLE SOFTWARE REQUIREMENTS SPECIFICATION

Web-Based Interactive Audio Visualizer and Digital Equalizer Platform

Real-Time Audio Spectrum Visualization & Multi-Band Equalization

Document Version 1.0
August 2026
Prepared for College/University Software Engineering Coursework

Technology Stack: HTML5 • CSS3 • JavaScript (ES6+) • Web Audio API • Canvas API
Client-Side Web Application — No Server Dependency for Core Features

Table of Contents

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document describes the functional and non-functional requirements for the **Web-Based Interactive Audio Visualizer and Digital Equalizer Platform**. It is intended to provide a complete and unambiguous understanding of the system for students, faculty evaluators, and future developers. The document follows the general structure recommended by the IEEE 830 / IEEE 29148 standards for software requirements specifications, adapted to a scope appropriate for a college-level software engineering project.

1.2 Document Conventions

Requirements are uniquely identified using the prefixes **FR** (Functional Requirement) and **NFR** (Non-Functional Requirement) followed by a sequential number, e.g. FR-1, NFR-3. Priority levels follow the MoSCoW convention: **Must Have**, **Should Have**, and **Could Have**. Technical terms are expanded on first use and summarized in the Glossary (Section 10).

1.3 Intended Audience and Reading Suggestions

Audience	Recommended Focus Sections
Project Guide / Faculty Evaluator	Sections 1, 2, 3, 5, 8, 9 - scope, features, quality attributes, testing
Student Development Team	Sections 3, 4, 6, 7 - functional requirements, interfaces, architecture, data design
UI/UX Designer	Sections 2.2, 3, 4.1 - features, user interface requirements
Software Tester / QA	Sections 3, 5, 9 - requirements, quality attributes, test plan

1.4 Project Scope

The system is a **responsive, client-side web application** that runs entirely inside a modern web browser using **HTML5 Canvas** for rendering and the **Web Audio API** for digital signal processing. Users can load a local audio file (or, as a stretch goal, stream from a microphone), see a real-time animated visualization of the audio spectrum, and shape the sound using a multi-band graphic equalizer. The project is scoped to be achievable by a small student team (2-4 members) within a single academic semester, while remaining architecturally sound enough to extend with a backend (accounts, cloud presets, sharing) in a future phase.

In scope for the current phase:

- Local audio file playback (MP3, WAV, OGG) through the browser.
- Real-time frequency-domain visualization rendered on HTML5 Canvas (bar, waveform, and circular/radial modes).
- A 10-band graphic equalizer built on Web Audio **BiquadFilterNode** objects.
- Playback controls: play, pause, seek, volume, mute.
- Theme customization (dark navy default theme, adjustable accent colors, visualization sensitivity).
- Saving and loading EQ presets locally using the browser's LocalStorage.
- Fully responsive layout for desktop, tablet, and mobile screens.

Out of scope for the current phase (documented as future work in Section 7):

- User authentication, cloud accounts, and server-side storage of presets.
- Real-time multi-user collaboration or social sharing of visualizations.
- Audio format conversion, editing, or export/rendering of processed audio files.

- Native mobile applications (iOS/Android) - the web app is responsive but not packaged natively.

1.5 Definitions, Acronyms and Abbreviations

Term	Definition
FFT	Fast Fourier Transform - converts a time-domain audio signal into frequency-domain data used for visualization.
Web Audio API	A browser JavaScript API for processing and synthesizing audio in web applications.
AudioContext	The core Web Audio API interface representing an audio-processing graph.
AnalyserNode	A Web Audio node that exposes real-time frequency and time-domain analysis data (FFT output).
BiquadFilterNode	A Web Audio node implementing a second-order (biquad) filter, used to build each equalizer band.
Canvas API	An HTML5 API for drawing 2D graphics via JavaScript, used to render the visualizer.
SRS	Software Requirements Specification.
EQ	Equalizer - a tool that boosts or cuts specific frequency bands of an audio signal.
LocalStorage	A browser-provided key-value storage mechanism that persists data on the client device.
FPS	Frames Per Second - rate at which the visualization canvas is redrawn.

2. Overall Description

2.1 Product Perspective

The platform is a **standalone, self-contained web application**. It does not depend on any existing product line and has no mandatory backend for its core functionality - all audio decoding, filtering, and rendering happen inside the user's browser using native Web APIs. This makes the system easy to deploy (static hosting, e.g. GitHub Pages / Netlify / Vercel) and ideal for an academic project timeline. Figure 1 shows how the application is internally layered.

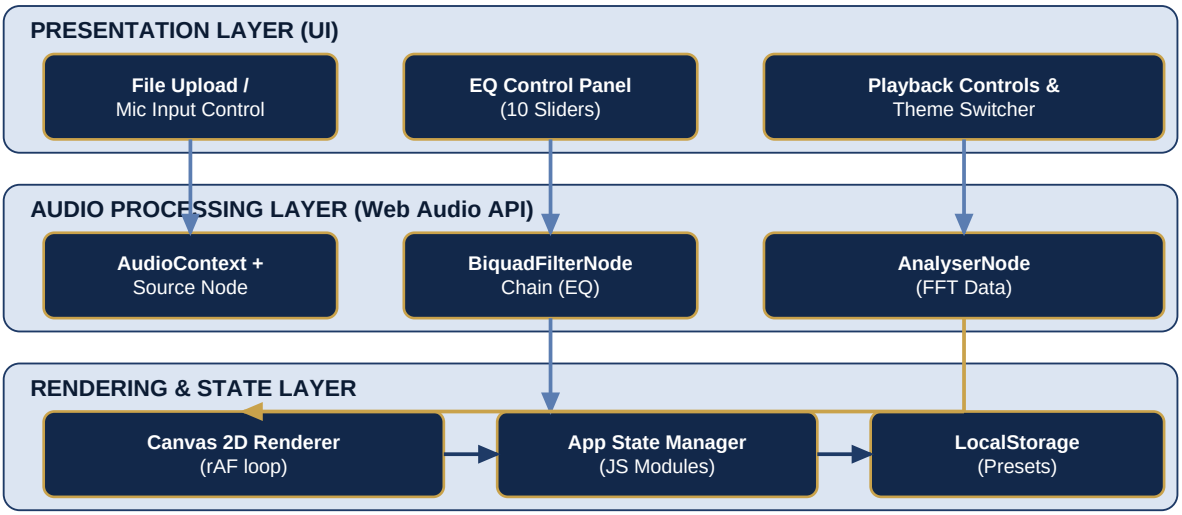

Figure 1 — Three-Layer System Architecture

2.2 Product Features (Summary)

- **Real-Time Spectrum Visualizer** — animated bar / waveform / radial views driven by live FFT data.
- **10-Band Graphic Equalizer** — interactive sliders mapped to BiquadFilterNode gain values.
- **Playback Engine** — play, pause, seek, volume/mute, track time display.
- **Preset Manager** — save, rename, delete, and re-apply custom EQ presets locally.
- **Theme & Style Switcher** — dark navy theme by default with adjustable accent colors and visual modes.
- **Responsive UI** — adapts layout for desktop, tablet, and mobile viewports.

2.3 User Classes and Characteristics

User Class	Description	Technical Skill
General Listener	Plays personal audio files and enjoys the visual effects.	Low
Audio Enthusiast	Fine-tunes EQ bands and saves custom presets for different genres.	Medium
Evaluator / Faculty	Reviews functionality, code quality, and requirement traceability.	High
Developer (future phase)	Extends the app with backend services, accounts, or new visual modes.	High

2.4 Operating Environment

Component	Requirement
Client Browser	Latest two versions of Google Chrome, Mozilla Firefox, Microsoft Edge, or Safari (Web Audio API & Canvas API support required).
Client Device	Desktop, laptop, tablet, or smartphone with audio output capability.
Hosting (deployment)	Any static web host (e.g. GitHub Pages, Netlify, Vercel) — no application server required for the current phase.
Network	Not required for core functionality (fully offline-capable once loaded); required only for initial page load.

2.5 Design and Implementation Constraints

- Must be implemented using only standard web technologies: HTML5, CSS3, and vanilla JavaScript (ES6+) or a lightweight framework, without requiring server-side code for core features.
- Audio processing must use the native **Web Audio API**; no proprietary or paid audio SDKs.
- Visualization must be rendered with the **HTML5 Canvas 2D API** for compatibility and performance.
- The system must function correctly without an internet connection once the static assets are loaded.
- Development timeline is constrained to a single academic semester with a small student team.

2.6 Assumptions and Dependencies

- Users have a modern, up-to-date web browser with JavaScript enabled.
- Audio files used for testing are in common formats (MP3/WAV/OGG) and are not DRM-protected.
- Microphone-based visualization (stretch goal) depends on the user granting browser microphone permission.
- The browser’s LocalStorage is available and not disabled/cleared by the user between sessions.

3. System Features (Functional Requirements)

This section details each functional module of the system. Every requirement is assigned a unique ID and priority to support requirement traceability during development and testing.

3.1 Audio Input and File Loading

ID	Requirement	Priority
FR-1.1	The system shall allow the user to select and load a local audio file (MP3, WAV, OGG) via a file picker or drag-and-drop.	Must
FR-1.2	The system shall decode the selected file into an AudioBuffer using the Web Audio API before playback.	Must
FR-1.3	The system shall display the file name and duration once loaded successfully.	Should
FR-1.4	The system shall show a clear error message if the selected file is invalid or unsupported.	Must
FR-1.5	The system shall optionally support live microphone input as an alternate audio source.	Could

3.2 Real-Time Spectrum Visualization

ID	Requirement	Priority
FR-2.1	The system shall use an AnalyserNode to extract frequency-domain (FFT) data from the audio stream in real time.	Must
FR-2.2	The system shall render the frequency data on an HTML5 Canvas at a minimum of 30 FPS, targeting 60 FPS.	Must
FR-2.3	The system shall provide at least three visualization modes: vertical bar spectrum, waveform (oscilloscope), and circular/radial spectrum.	Must
FR-2.4	The system shall allow the user to switch between visualization modes without interrupting playback.	Should
FR-2.5	The system shall allow adjustment of visual sensitivity/gain and color theme of the visualization.	Should
FR-2.6	The canvas shall automatically resize to fit the browser window/container on resize events.	Must

3.3 Multi-Band Digital Equalizer

ID	Requirement	Priority
FR-3.1	The system shall provide 10 equalizer bands, each implemented as a chained BiquadFilterNode (peaking filter) centered at standard ISO frequencies (e.g., 31, 62, 125, 250, 500, 1k, 2k, 4k, 8k, 16k Hz).	Must
FR-3.2	The system shall provide a vertical slider control per band, allowing gain adjustment typically from -12 dB to +12 dB.	Must
FR-3.3	The system shall apply gain changes to the connected filter node(s) in real time with no audible glitching.	Must
FR-3.4	The system shall provide a one-click 'Reset EQ' action that returns all bands to 0 dB (flat).	Should
FR-3.5	The system shall provide built-in genre presets (e.g., Rock, Pop, Jazz, Bass Boost, Vocal Boost, Flat).	Should

3.4 Playback Controls

ID	Requirement	Priority
FR-4.1	The system shall provide play and pause controls for the loaded audio.	Must
FR-4.2	The system shall provide a seek bar reflecting current playback position, with click/drag-to-seek support.	Must
FR-4.3	The system shall provide a volume slider and a mute toggle.	Must
FR-4.4	The system shall display elapsed time and total duration in MM:SS format.	Should
FR-4.5	The system shall support keyboard shortcuts for common actions (space = play/pause, arrows = seek/volume).	Could

3.5 Preset Management (Local Persistence)

ID	Requirement	Priority
FR-5.1	The system shall allow the user to save the current EQ configuration as a named custom preset.	Should
FR-5.2	The system shall persist saved presets in the browser's LocalStorage so they are available on the next visit.	Should
FR-5.3	The system shall allow the user to load, rename, or delete a saved preset.	Should
FR-5.4	The system shall confirm destructive actions (e.g., delete preset) before executing them.	Should

3.6 Theming, Customization and Responsiveness

ID	Requirement	Priority
FR-6.1	The system shall load with a Dark Navy theme by default, applied consistently across all UI panels.	Must
FR-6.2	The layout shall reflow responsively across breakpoints for desktop (≥1024px), tablet (768-1023px), and mobile (<768px).	Must
FR-6.3	The system shall allow the user to change the visualizer's accent color independent of the base theme.	Could
FR-6.4	All interactive controls (sliders, buttons) shall provide visible hover/focus/active states.	Should

4. External Interface Requirements

4.1 User Interfaces

The UI is organized into four primary regions: (1) a top navigation/header bar with file load and theme controls, (2) a central canvas panel that fills most of the viewport for the visualizer, (3) a bottom playback control bar (play/pause, seek, volume), and (4) a collapsible side/bottom panel containing the 10-band equalizer sliders and preset manager. On mobile, panels (3) and (4) collapse into a swipeable bottom sheet to preserve screen space for the visualization.

Screen / Panel	Purpose
Header Bar	App title, file/mic input button, theme toggle.
Visualizer Canvas	Full-size real-time rendering area; mode switch buttons overlay top-right.
Playback Bar	Play/Pause, seek bar with time labels, volume, mute.
Equalizer Panel	10 vertical gain sliders, preset dropdown, save/reset buttons.

4.2 Hardware Interfaces

The application interfaces with the client device’s audio output hardware (speakers/headphones) via the browser’s default audio output, and optionally with the device’s microphone hardware for the live-input stretch goal, mediated entirely through browser-provided APIs (no direct hardware access).

4.3 Software Interfaces

Interface	Description
Web Audio API	Browser-native API used for decoding, filtering (EQ), gain control, and FFT analysis.
Canvas 2D API	Browser-native API used to render the visualizer frames.
File API	Browser-native API used to read local audio files selected by the user.
Web Storage API (LocalStorage)	Used to persist EQ presets and theme preference on the client.
MediaDevices API (optional)	Used to request microphone access for live-input visualization.

4.4 Communication Interfaces

The current phase requires no network communication for core functionality beyond the initial static asset download (HTML/CSS/JS) over HTTPS. Section 7 describes a future REST API for cloud-synced presets and accounts.

5. Non-Functional Requirements

5.1 Performance Requirements

ID	Requirement
NFR-1	The visualizer shall maintain a minimum of 30 FPS (target 60 FPS) on a mid-range laptop/desktop for standard visualization modes.
NFR-2	Audible latency introduced by the EQ filter chain shall not exceed ~20 ms, remaining imperceptible to the user.
NFR-3	The application shall load and become interactive within 3 seconds on a broadband connection.
NFR-4	Switching visualization modes or applying an EQ preset shall complete within 200 ms.

5.2 Usability

ID	Requirement
NFR-5	First-time users shall be able to load a file and hear/see the visualizer without external instructions (self-explanatory UI).
NFR-6	All controls shall include accessible labels/tooltips and sufficient color contrast against the dark navy theme.
NFR-7	The interface shall remain fully operable using keyboard navigation for critical controls.

5.3 Reliability and Availability

ID	Requirement
NFR-8	The system shall handle unsupported/corrupted audio files gracefully with a descriptive error, without crashing the page.
NFR-9	As a static client-side application, the system shall be available whenever the hosting page is reachable, with no server downtime risk for core features.

5.4 Security and Privacy

ID	Requirement
NFR-10	The system shall not transmit user audio files or EQ presets to any external server in the current phase; all processing stays on-device.
NFR-11	Microphone access (if used) shall only be requested with explicit user action and a visible active-recording indicator.

5.5 Maintainability and Portability

ID	Requirement
NFR-12	Code shall be organized into clearly separated JavaScript modules (audio engine, visualizer, EQ, UI controls, storage) to support maintainability.
NFR-13	The application shall run correctly on the latest two versions of Chrome, Firefox, Edge, and Safari without polyfills for core features.
NFR-14	The codebase shall follow a consistent style guide (e.g., ESLint/Prettier) enabling future contributors to onboard quickly.

6. System Architecture and Design Overview

The application follows a modular, layered client-side architecture (see Figure 1). The Audio Processing Layer builds a Web Audio graph that chains the source through a gain node, a series of ten BiquadFilterNode EQ bands, and an AnalyserNode, before reaching the destination (speakers). Figure 2 illustrates this audio signal flow, which is the technical heart of the system.

Figure 2 — Web Audio API Signal Processing Graph

6.1 Module Breakdown

Module	Responsibility
audio-engine.js	Manages AudioContext, source node creation (file/mic), gain node, and the EQ filter chain.
equalizer.js	Defines the 10 band frequencies/Q values, exposes setBandGain(), and links UI sliders to filter nodes.
visualizer.js	Reads FFT/time-domain data from the AnalyserNode each animation frame and draws the selected visualization mode on canvas.
ui-controller.js	Wires up DOM events for playback controls, mode switching, and theme toggling.
preset-manager.js	Serializes/deserializes EQ presets to and from LocalStorage.
app.js	Application entry point; initializes modules and manages global state.

6.2 Suggested Project Folder Structure

```
audio-visualizer/
  index.html
  /css     style.css, theme-navy.css
  /js
    audio-engine.js
    equalizer.js
    visualizer.js
    ui-controller.js
    preset-manager.js
    app.js
  /assets   icons, sample audio
  /docs     SRS, diagrams, test plan
```

7. Data Requirements and Future API / Database Planning

The current phase intentionally avoids a backend, keeping the project achievable within a semester. However, the design anticipates a Phase 2 in which user accounts and cloud-synced presets are introduced. This section documents that forward-looking plan for evaluators interested in scalability and system design maturity.

7.1 Current Phase: Local Data Model

EQ presets are stored client-side as JSON objects in LocalStorage under a namespaced key, e.g.:

```
{ "id": "preset-1699", "name": "Bass Boost", "bands": [6, 4, 2, 0, -1, -1, 0, 1, 2, 3],
"createdAt": "2026-08-01" }
```

7.2 Level-0/1 Data Flow

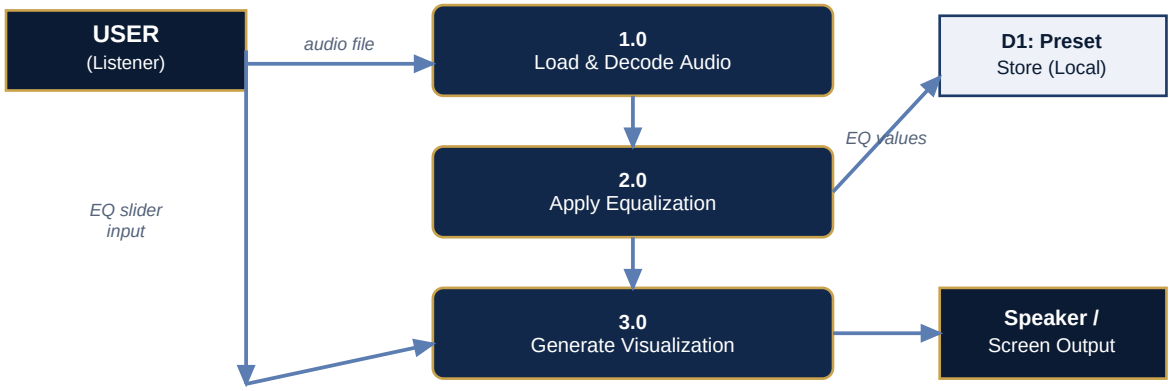

Figure 3 — Simplified Data Flow Diagram

7.3 Future Phase: Proposed Relational Schema

Table	Key Columns
users	user_id (PK), username, email, password_hash, created_at
presets	preset_id (PK), user_id (FK), name, band_values (JSON), is_public, created_at
visualization_settings	setting_id (PK), user_id (FK), theme, default_mode, sensitivity

7.4 Future Phase: Proposed REST API

Endpoint	Method	Description
/api/auth/register	POST	Create a new user account.
/api/auth/login	POST	Authenticate a user and issue a session token.
/api/presets	GET	Retrieve all presets belonging to the authenticated user.
/api/presets	POST	Create a new cloud-synced preset.
/api/presets/:id	PUT / DELETE	Update or remove a specific preset.

Suggested future stack: Node.js + Express for the API layer, MongoDB or PostgreSQL for persistence, and JWT-based authentication - kept deliberately decoupled from the client so the current client-side application requires

no code changes to its audio/visual core when this phase is introduced.

8. Use Case Model

The primary actor is the **Listener/User**. Figure 4 (described textually below in place of a UML diagram for print clarity) and Table 8.1 summarize the core use cases.

- UC-1: Load Audio File
- UC-2: Play / Pause / Seek Track
- UC-3: Adjust Equalizer Band
- UC-4: Save / Load EQ Preset
- UC-5: Switch Visualization Mode
- UC-6: Change Theme / Accent Color
- UC-7: Enable Microphone Input (stretch)

8.1 Detailed Use Case: Adjust Equalizer Band

Field	Description
Use Case ID	UC-3
Actor	Listener / Audio Enthusiast
Preconditions	An audio file is loaded and the EQ panel is visible.
Main Flow	1. User drags a band slider. 2. UI captures the new gain value. 3. equalizer.js calls setBandGain() on the corresponding BiquadFilterNode. 4. The audio output updates in real time. 5. The visualizer reflects the altered frequency response on the next frame.
Alternate Flow	User clicks 'Reset EQ' -> all bands set to 0 dB in one action.
Postconditions	The audio graph reflects the new EQ curve; state can optionally be saved as a preset (UC-4).

8.2 Detailed Use Case: Load Audio File

Field	Description
Use Case ID	UC-1
Actor	Listener
Preconditions	The application has finished initial load.
Main Flow	1. User clicks 'Load File' or drags a file onto the drop zone. 2. Browser File API reads the file. 3. audio-engine.js decodes it via AudioContext.decodeAudioData(). 4. On success, playback controls and the visualizer become active.
Alternate Flow	Invalid/corrupt file -> system shows an inline error message and remains on the current state.
Postconditions	The decoded AudioBuffer is connected to the Web Audio graph and ready for playback.

9. Software Testing Requirements

Testing shall cover functional correctness, cross-browser compatibility, performance under load, and usability. The table below lists representative test cases traceable to the functional requirements in Section 3.

Test ID	Requirement	Test Description	Expected Result
TC-01	FR-1.1/1.2	Upload a valid MP3 file.	File decodes and playback controls activate.
TC-02	FR-1.4	Upload an unsupported file type (e.g., .txt).	System shows a clear error, no crash.
TC-03	FR-2.2/2.3	Play audio and switch between bar, waveform, and radial modes.	Canvas updates smoothly (≥30 FPS) in each mode.
TC-04	FR-3.2/3.3	Drag a band slider to +10 dB while playing.	Audible change occurs within ~20 ms, no glitching.
TC-05	FR-3.5	Apply the 'Bass Boost' preset.	All band sliders animate to the preset's stored values.
TC-06	FR-4.1/4.2	Click play, then drag the seek bar mid-track.	Playback resumes accurately from the new position.
TC-07	FR-5.1/5.2	Save a custom preset, reload the page.	The saved preset persists and reloads correctly.
TC-08	FR-6.2	Resize browser from desktop to mobile width.	Layout reflows without overlapping/broken elements.
TC-09	NFR-13	Run the app on Chrome, Firefox, and Edge.	Consistent behavior and visuals across browsers.
TC-10	NFR-8	Load a corrupted audio file.	Graceful error message; app remains usable.

9.1 Testing Levels

- **Unit Testing** — individual functions (e.g., setBandGain, preset serialization) tested in isolation.
- **Integration Testing** — verifying the audio graph (source → EQ → analyser → destination) behaves correctly end-to-end.
- **System Testing** — full user flows from file load through visualization and preset save/reload.
- **Usability Testing** — informal walkthroughs with classmates to validate UI clarity ahead of faculty demo.
- **Cross-Browser Testing** — manual verification on Chrome, Firefox, Edge, and Safari.

10. Appendices

10.1 Traceability Summary

Every functional requirement in Section 3 maps to at least one test case in Section 9 and to a module described in Section 6.1, ensuring end-to-end traceability from requirement to design to verification - a key expectation for faculty evaluation of the software engineering process.

10.2 Assumptions Recap

- The evaluation environment provides a modern desktop or laptop browser for the primary demo.
- Sample audio files for the demo are royalty-free or the team's own recordings.
- The project is developed and version-controlled using Git, with incremental commits mapped to milestones.

10.3 Suggested Development Milestones

Milestone	Deliverable
M1 - Setup & Audio Engine	Project scaffold, AudioContext setup, file load + basic playback.
M2 - Visualizer Core	AnalyserNode integration; bar visualization mode on canvas.
M3 - Equalizer	10-band BiquadFilterNode chain wired to UI sliders.
M4 - Presets & Extra Visual Modes	Waveform & radial modes; LocalStorage preset save/load.
M5 - Theming, Responsiveness & Testing	Dark navy theme polish, mobile layout, full test pass, documentation.

10.4 Document Approval

Role	Name	Date
Project Student(s)	Vaib	August 2026
Project Guide / Faculty		
Reviewer (optional)		